

SuVa Protocol — Technical Overview & Academic Collaboration Proposal

Prepared by: William Schulz, UCL Alumni 2023 **Date:** January 2026 **Status:** Pre-audit, experimental **Website:** www.suva.tech

1. Executive Summary

This document describes the codebase and roadmap for **SuVa**, a quantum-proof stablecoin protocol for Ethereum Virtual Machine (EVM) blockchains.

SuVa's goal is to provide the cryptographic infrastructure that allows stablecoin holders to protect their assets against the quantum computing threat. The protocol wraps existing stablecoins (USDT, USDC, etc.) into quantum-secured versions using NIST-standardised post-quantum signature schemes.

The protocol's cryptographic strategy is a **dual-scheme approach**:

- **Falcon (FN-DSA / FIPS 206)** — Primary on-chain verifier. Lattice-based (NTRU). Extremely gas-efficient (~350K gas per verification). Used for daily transactions — cheap, fast, and quantum-resistant.
- **SPHINCS+ (SLH-DSA / FIPS 205)** — Secondary "nuclear bunker" option. Hash-based (no lattice assumptions). More expensive on-chain but relies only on hash functions — the oldest and most trusted cryptographic primitive. Used for high-value or long-term security where maximum protection is required.

This dual approach ensures that even if lattice-based cryptography is ever compromised, SPHINCS+ provides an independent security guarantee based on entirely different mathematical foundations.

The current codebase implements **Dilithium (ML-DSA / FIPS 204)** on-chain signature verification, which serves as a research foundation. The roadmap prioritises migrating to **Falcon** for production use due to its superior gas efficiency, while adding **SPHINCS+** as the conservative fallback.

We are seeking academic collaborators to validate, guide, and co-develop this infrastructure.

2. The Quantum Threat to Blockchain

2.1 Why This Matters Now

All current Ethereum transactions are secured by **ECDSA** (Elliptic Curve Digital Signature Algorithm). Shor's algorithm, running on a sufficiently powerful quantum computer, can break ECDSA in polynomial time — meaning every wallet, every stablecoin contract, and every DeFi protocol on Ethereum becomes vulnerable.

NIST timeline:

- **2030** — ECDSA and RSA deprecated for all US government use
- **2035** — ECDSA and RSA banned entirely

Stablecoin exposure: As of January 2026, over **\$200 billion** in stablecoins (USDT, USDC, DAI, etc.) are secured solely by ECDSA signatures. There is currently no production-ready post-quantum signature infrastructure for EVM blockchains.

2.2 "Harvest Now, Decrypt Later"

Adversaries can record encrypted blockchain transactions today and decrypt them once quantum computers are available. This is known as the "harvest now, decrypt later" attack. For long-lived assets like stablecoins (which may be held for years), the quantum threat is not a future problem — it is a present data collection risk.

2.3 What Needs to Change

To protect stablecoin assets, the signature scheme used to authorise transactions must be replaced with a quantum-resistant alternative. This requires:

1. **On-chain verifiers** — Smart contracts that can verify post-quantum signatures (Falcon, SPHINCS+, Dilithium)
2. **Off-chain signing tools** — Software for users to generate post-quantum keypairs and sign transactions
3. **Migration infrastructure** — Contracts that allow users to move existing assets into quantum-safe custody

SuVa provides all three layers.

3. What Exists Today

3.1 Dilithium Verification on EVM (Built by ZKNox)

The current codebase is an open-source (MIT licensed) implementation of Dilithium signature verification for the Ethereum Virtual Machine, developed by **ZKNox (Renaud Dubois, Simon Masson)**. SuVa has identified, evaluated, and selected this implementation as the foundation for its protocol.

The codebase provides:

- **On-chain signature verification** in Solidity (smart contracts)
- **Off-chain key generation and signing** in Python (reference implementation)
- **Two verification variants**: standard Dilithium (NIST-compliant) and ETHDilithium (gas-optimised)

3.2 Languages Used

Language	Purpose	Version
Solidity	On-chain signature verification (smart contracts)	^0.8.25
Python	Off-chain key generation, signing, and test vector generation	3.12
Yul (EVM Assembly)	Inline assembly for gas-critical operations (NTT, polynomial arithmetic)	Embedded in Solidity

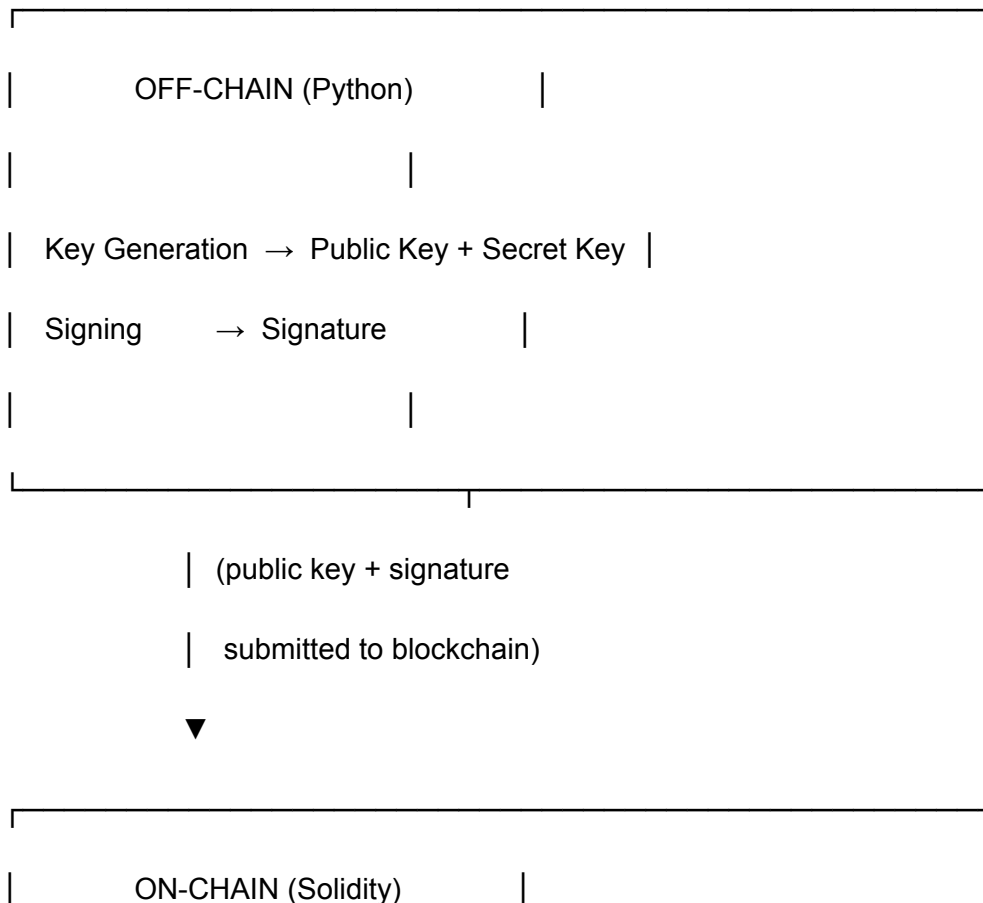
Frameworks and tools:

- **Foundry (Forge)** — Solidity testing framework
- **SHAKE-256 / SHAKE-128** — Extendable output functions (hash functions used in Dilithium)
- **AES-256 CTR DRBG** — Deterministic random byte generator for reproducible test vectors

3.3 Codebase Size

Component	Files	Lines of Code	Author
Solidity contracts (core)	9	~1,600	ZKNox
Solidity tests	6	~3,200	ZKNox
Python reference implementation	~20	~3,900	Forked from Giacomo Pope, modified by ZKNox
Security estimates (research)	6	~900	ZKNox (Simon Masson)
TOTAL	~41	~9,800 lines	

3.4 Architecture Overview



Verification	→ true / false
(runs inside EVM smart contract)	

4. Detailed Code Description

4.1 Solidity Smart Contracts (On-Chain Verification)

File	Lines	Description
<code>ZKNOX_dilithium.sol</code>	171	Standard Dilithium verifier. Implements the full NIST-compliant verification algorithm (FIPS 204). Uses SHAKE-256 for all hashing. Gas cost: ~13.5M. This is the entry point that stablecoin contracts would call.
<code>ZKNOX_ethdilithium.sol</code>	124	Gas-optimised verifier. Replaces SHAKE-256 with Keccak-256 (native EVM opcode, much cheaper) and uses precomputed NTT values embedded in the public key. Gas cost: ~6.6M (~50% reduction). Not strictly NIST-compliant due to hash substitution.
<code>ZKNOX_dilithium_core.sol</code>	103	Core verification logic. Split into two steps: <code>dilithium_core_1()</code>

File	Lines	Description
		validates hint vector norms and expands signature components (z, h); <code>dilithium_core_2()</code> computes the matrix-vector product $A \cdot z - c \cdot t_1$ and recovers w' (the high-order bits needed for final hash comparison).
<code>ZKNOX_NTT.sol</code>	269	Number Theoretic Transform (NTT). Forward and inverse NTT for polynomial multiplication over F_q . Written entirely in Yul assembly for gas efficiency. Uses <code>extcodecopy</code> to read precomputed twiddle factors from separately deployed contracts (reducing runtime computation). This is the most gas-critical component.
<code>ZKNOX_NTT_dilithium.sol</code>	151	Dilithium-specific NTT wrappers. Adapts the generic NTT to Dilithium parameters ($n=256$ coefficients, prime $q=8380417$). Provides <code>ZKNOX_NTTFW()</code> (forward) and <code>ZKNOX_NTTINV()</code> (inverse) entry points.
<code>ZKNOX_dilithium_utils.sol</code>	231	Constants, data structures, and utility functions. Defines all Dilithium parameters ($n, q, k, l, \text{eta}, \text{tau}, \text{gamma}, \text{omega}, d, \text{beta}$). Declares <code>Signature</code> and <code>PubKey</code> structs. Implements vectorised modular

File	Lines	Description
		arithmetic: <code>VECMULMOD</code> , <code>VECADDMOD</code> , <code>VECSUBMOD</code> , <code>ScalarProduct</code> , <code>MatVecProduct</code> .
<code>ZKNOX_hint.sol</code>	145	Hint reconstruction. Implements <code>useHint()</code> to recover high-order bits of <code>w</code> from the hint vector <code>h</code> and the computed value <code>Az - ct1</code> . The hint is a compact encoding that avoids transmitting the full <code>w</code> vector.
<code>ZKNOX_shake.sol</code>	269	SHAKE-256 implementation. Full software implementation of the SHAKE extendable output function (XOF) in Solidity. Required for NIST-compliant Dilithium but expensive on EVM (~50% of total gas). This is why ETHDilithium replaces it with native Keccak.
<code>ZKNOX_keccak_prng.sol</code>	20	Keccak-based PRNG. Lightweight pseudo-random number generator using Ethereum's native Keccak-256 opcode (<code>keccak256</code>). Used in ETHDilithium as a gas-cheap replacement for SHAKE-256 when expanding the matrix <code>A</code> and generating challenges.
<code>ZKNOX_dilithium_deploy.sol</code>	128	Deployment helpers. Deploys precomputed twiddle factor contracts (constants needed by the NTT) and

File	Lines	Description
		initialises the verifier with their addresses. Twiddle factors are stored as contract bytecode and read via <code>extcodecopy</code> at verification time.

4.2 Python Reference Implementation (Off-Chain Signing)

File / Module	Description
<code>Falcon_dilithium.py</code>	Core Dilithium class (~320 lines). Implements the complete Dilithium lifecycle: <code>keygen()</code> generates public/secret keypairs, <code>sign()</code> produces signatures via rejection sampling, <code>verify()</code> checks signatures. Also handles packing/unpacking of keys and signatures to/from byte format for transmission.
<code>Falcon_dilithium_eth.py</code>	ETHDilithium variant . Modified version that replaces SHAKE-256 with Keccak-256 (matching the gas-optimised Solidity verifier) and precomputes NTT values for the public key. Signatures from this variant are verified by <code>ZKNOX_ethdilithium.sol</code> .
<code>Falcon_default_parameters.py</code>	Parameter sets . Defines all Dilithium2 constants: $n=256$, $q=8380417$, $k=4$, $l=4$, $\eta=2$, $\tau=39$, $\gamma_1=2^{17}$, $\gamma_2=(q-1)/88$, $d=13$, $\omega=80$. These must match the Solidity contract constants exactly.
<code>polynomials/</code>	Polynomial arithmetic (multiple files). Ring operations over $\mathbb{Z}_q[X]/(X^{256} + 1)$: NTT forward/inverse transforms, coefficient reduction modulo q , infinity norm checking, power-2 rounding, decompose/make_hint operations, and bit-packing for compact byte encoding.

File / Module	Description
<code>modules/</code>	Module (vector/matrix) operations. Matrix-vector multiplication ($A * s_1$), vector addition ($t = As_1 + s_2$), scalar multiplication ($c * s_1$), and element-wise operations — all over the polynomial ring. Used to build the lattice-based algebraic structure.
<code>shake/</code>	SHAKE wrapper. Absorb/squeeze interface for SHAKE-128 (used to expand matrix A) and SHAKE-256 (used for challenge hash, message hash, and seed expansion). Wraps Python's <code>hashlib.shake_256</code> .
<code>Falcon_drbg/</code>	Deterministic RNG. AES-256 CTR DRBG (Deterministic Random Bit Generator) for reproducible key generation. Used to align with NIST Known Answer Test (KAT) vectors — ensures that given the same seed, the same keys and signatures are produced. Requires <code>pycryptodome</code> .
<code>generate_dilithium_test_vectors.py</code>	Dilithium test vector generator. Runs the Python signing code and outputs Solidity-compatible test data (public key, message, signature as hex-encoded bytes) that can be fed into Forge tests to verify cross-language correctness.
<code>generate_ethdilithium_test_vectors.py</code>	ETHDilithium test vector generator. Same as above but for the gas-optimised Keccak variant. Produces test vectors compatible with <code>ZKNOX_ethdilithium.sol</code> .

4.3 Security Estimates (Research)

These Python scripts estimate the concrete security level of Dilithium's underlying lattice problems. They model the cost of the best known attacks (lattice reduction via BKZ algorithm) to confirm that the chosen parameters achieve NIST Security Level 2 (equivalent to AES-128 security).

File	Description
<code>Falcon_Dilithium.py</code>	Security analysis of standard Dilithium parameters — computes estimated bit-security for the Dilithium2 parameter set
<code>Falcon_ZKDilithium.py</code>	Analysis of a ZK-friendly variant using an alternative prime field ($q = 2013265921$, an NTT-friendly prime). Explores whether a different modulus could enable ZK-proof compatibility.
<code>Falcon_MLWE_security.py</code>	Module Learning With Errors (MLWE) hardness estimates. MLWE is the core hard problem underlying Dilithium's security — recovering s_1 from $(A, A \cdot s_1 + s_2)$ must be computationally infeasible.
<code>Falcon_MSIS_security.py</code>	Module Short Integer Solution (MSIS) hardness estimates. MSIS ensures that forging a signature (finding a short vector in the lattice) is computationally infeasible.
<code>Falcon_model_BKZ.py</code>	BKZ lattice reduction cost model. Models the Block Korkin-Zolotarev algorithm, which is the best known attack against lattice-based cryptography. Estimates the number of operations needed to break the scheme.
<code>Falcon_proba_util.py</code>	Probability utilities. Helper functions for the security estimates (statistical distributions, tail bounds).

5. Cryptographic Parameters (Dilithium Level II — NIST Security Level 2)

Parameter	Value	Description
n	256	Polynomial degree
q	8,380,417	Prime modulus

Parameter	Value	Description
k	4	Number of rows in matrix A
l	4	Number of columns in matrix A
eta (η)	2	Secret key coefficient bound
tau (τ)	39	Number of ± 1 entries in challenge polynomial
gamma_1 (γ_1)	$2^{17} = 131,072$	Masking range for signature vector z
gamma_2 (γ_2)	$(q-1)/88 = 95,232$	Low-order rounding range
d	13	Dropped bits from public key
omega (ω)	80	Maximum number of 1s in hint vector h
beta (β)	$\tau \times \eta = 78$	Norm bound for rejection sampling

6. Gas Benchmarks — Post-Quantum Scheme Comparison

Scheme	Gas Cost (EVM)	Signature Size	Security Basis	NIST Standard	Role in SuVa
ECDSA (current Ethereum)	~3K gas	65 bytes	Elliptic curve	—	Broken by quantum
Falcon	~350K gas	~690 bytes	Lattice (NTRU)	FIPS 206	Primary — daily use
ETHDilithium	~6.6M gas	~2,420 bytes	Lattice (Module-LWE)	FIPS 204 (modified)	Research foundation

Scheme	Gas Cost (EVM)	Signature Size	Security Basis	NIST Standard	Role in SuVa
Dilithium	~13.5M gas	~2,420 bytes	Lattice (Module-LWE)	FIPS 204	Research foundation
SPHINCS+	Estimated 50-500M+ gas	~7,856 - 49,856 bytes	Hash-based	FIPS 205	Fallback — maximum security

Key insight: Falcon is ~20x cheaper than Dilithium and ~100-1000x cheaper than SPHINCS+ on EVM. This is why SuVa's production strategy uses Falcon as the primary scheme.

Ethereum block gas limit: 30M. Falcon fits easily (~350K). On L2 chains (Arbitrum, Optimism, Base), gas costs are 10-100x lower, making even Dilithium and potentially SPHINCS+ practical.

ZKNox has already built both Falcon (ETHFALCON) and Dilithium (ETHDILITHIUM) for EVM. The Falcon implementation is available at <https://github.com/ZKNoxHQ/ETHFALCON>.

7. Code Fragment — On-Chain Verification (Solidity)

The following is the core signature verification function from `ZKNOX_dilithium.sol`. This is the function that would be called on-chain by stablecoin contracts to verify a post-quantum signature. The corresponding off-chain signing code is in Section 8.

```
function verify(
    PubKey memory pk,
    bytes memory msgs,
    Signature memory signature
) external view returns (bool) {
    // STEP 1: Validate signature components
    (uint256 norm_h, uint256[] memory h, uint256[] memory z) =
```

```

    dilithium_core_1(signature);

if (norm_h > omega) { return false; }

// Check z coefficients are within bound

for (uint i = 0; i < 4; i++) {

    for (uint j = 0; j < 256; j++) {

        if (z[i][j] > gamma_1_minus_beta

            && 8380417 - z[i][j] > gamma_1_minus_beta) {

            return false;

        }

    }

}

// STEP 2: Reconstruct challenge polynomial c in NTT domain

uint256[] memory c_ntt = compute_c_ntt(signature);

// STEP 3: Compute (1 << d) * t1 in NTT domain

uint256[][] memory t1_new = ZKNOX_Expand_Vec(pk.t1);

for (uint i = 0; i < 4; i++) {

    for (uint j = 0; j < 256; j++) { t1_new[i][j] <= d; }

    t1_new[i] = ZKNOX_NTTFW(t1_new[i], apsirev);

}

// STEP 4: Compute w' = UseHint(h, A*z - c*t1)

bytes memory w_prime_bytes =

    dilithium_core_2(apsirev, apsilnvrev, pk, z, c_ntt, h, t1_new);

```

```

// STEP 5: Final hash comparison

ctx_shake memory ctx;

ctx = shake_update(ctx, pk.tr);

ctx = shake_update(ctx, msgs);

bytes memory mu = shake_digest(ctx, 64);

ctx_shake memory ctx2;

ctx2 = shake_update(ctx2, mu);

ctx2 = shake_update(ctx2, w_prime_bytes);

bytes32 final_hash = bytes32(shake_digest(ctx2, 32));

for (uint i = 0; i < 32; i++) {

    if (signature.c_tilde[i] != final_hash[i]) { return false; }

}

return true;

}

```

8. Code Fragment — Off-Chain Key Generation and Signing (Python)

The following shows the Python reference implementation used for off-chain key generation and signing. This code runs on the user's device (not on the blockchain). The generated signature is then submitted to the on-chain Solidity verifier (Section 6).

```

class Dilithium:

    def keygen(self):

```

```

# Generate random 32-byte seed

zeta = self.random_bytes(32)

# Expand seed using SHAKE-256 into three components:

# rho (32 bytes)    — seed for public matrix A

# rho_prime (64 bytes) — seed for secret vectors s1, s2

# K (32 bytes)      — seed for signing randomness

seed_bytes = self._h(zeta, 128)

rho, rho_prime, K = seed_bytes[:32], seed_bytes[32:96], seed_bytes[96:]

# Generate public matrix  $A \in R^{(k \times l)}$  in the NTT domain from rho

# A is a 4x4 matrix of polynomials, each of degree 255

A_hat = self._expand_matrix_from_seed(rho)

# Generate secret vectors  $s1 \in R^l$ ,  $s2 \in R^k$ 

# Coefficients bounded by eta ( $\pm 2$  for Dilithium2)

s1, s2 = self._expand_vector_from_seed(rho_prime)

# Compute  $t = A * s1 + s2$  (the public key vector)

t = (A_hat @ s1.to_ntt()).from_ntt() + s2

# Split t into t1 (high bits, published) and t0 (low bits, kept secret)

t1, t0 = t.power_2_round(self.d)

# Pack and return: public key = (rho, t1), secret key = (rho, K, tr, s1, s2, t0)

pk = self._pack_pk(rho, t1)

tr = self._h(pk, 32)

sk = self._pack_sk(rho, K, tr, s1, s2, t0)

```

```

return pk, sk

def sign(self, sk_bytes, m):

    # Unpack secret key components

    rho, K, tr, s1, s2, t0 = self._unpack_sk(sk_bytes)

    # Regenerate public matrix A

    A_hat = self._expand_matrix_from_seed(rho)

    # Compute message representative: mu = H(tr || message)

    mu = self._h(tr + m, 64)

    kappa = 0

    rho_prime = self._h(K + mu, 64)

    # Precompute NTT representations of secret vectors

    s1, s2, t0 = s1.to_ntt(), s2.to_ntt(), t0.to_ntt()

    # Rejection sampling loop — may iterate multiple times

    # until a valid signature is found

    alpha = self.gamma_2 << 1

    while True:

        # Sample masking vector y with coefficients in [-gamma_1, gamma_1]

        y = self._expand_mask_vector(rho_prime, kappa)

        kappa += self.l # increment nonce

        # Compute w = A * y

        w = (A_hat @ y.to_ntt()).from_ntt()

```



```

# Decompose w into high bits (w1) and low bits (w0)

w1, w0 = w.decompose(alpha)

# Create challenge polynomial c (exactly tau=39 coefficients are ±1)

c_tilde = self._h(mu + w1.bit_pack_w(self.gamma_2), 32)

c = self.R.sample_in_ball(c_tilde, self.tau).to_ntt()

# Compute z = y + c * s1 (the signature vector)

z = y + (s1.scale(c)).from_ntt()

# REJECTION: if z is too large, restart (prevents secret key leakage)

if z.check_norm_bound(self.gamma_1 - self.beta):

    continue

# Additional checks on w0 - c*s2 and c*t0 norms

w0_minus_cs2 = w0 - s2.scale(c).from_ntt()

if w0_minus_cs2.check_norm_bound(self.gamma_2 - self.beta):

    continue

c_t0 = t0.scale(c).from_ntt()

if c_t0.check_norm_bound(self.gamma_2):

    continue

# Compute hint vector h for verification

h = (w0_minus_cs2 + c_t0).make_hint_optimised(w1, alpha)

if h.sum_hint() > self.omega:

    continue

# All checks passed — return packed signature

```

```

        return self._pack_sig(c_tilde, z, h)

def verify(self, pk_bytes, m, sig_bytes):

    # Unpack public key and signature

    rho, t1 = self._unpack_pk(pk_bytes)

    c_tilde, z, h = self._unpack_sig(sig_bytes)

    # Check hint and z norms

    if h.sum_hint() > self.omega:

        return False

    if z.check_norm_bound(self.gamma_1 - self.beta):

        return False

    # Regenerate matrix A and challenge c

    A_hat = self._expand_matrix_from_seed(rho)

    tr = self._h(pk_bytes, 32)

    mu = self._h(tr + m, 64)

    c = self.R.sample_in_ball(c_tilde, self.tau).to_ntt()

    # Compute Az - ct1*(2^d) in NTT domain

    z_ntt = z.to_ntt()

    t1_scaled = t1.scale(1 << self.d).to_ntt()

    Az_minus_ct1 = ((A_hat @ z_ntt) - t1_scaled.scale(c)).from_ntt()

    # Use hint to recover w' (the high-order bits)

    w_prime = h.use_hint(Az_minus_ct1, 2 * self.gamma_2)

```

Final check: recompute challenge and compare

return c_tilde == self._h(mu + w_prime.bit_pack_w(self.gamma_2), 32)

How keygen, sign, and verify relate:

KEYGEN (off-chain, once):

seed $\rightarrow (A, s_1, s_2) \rightarrow t = A*s_1 + s_2 \rightarrow$ public key = (rho, t1)

SIGN (off-chain, per message):

secret key + message $\rightarrow$ rejection loop $\rightarrow$ signature = (c_tilde, z, h)

VERIFY (on-chain, Solidity):

public key + message + signature $\rightarrow$ recompute w' $\rightarrow$ compare hashes $\rightarrow$ true/false

9. Current Status

Item	Status
Dilithium on-chain verification (FIPS 204)	Complete (ZKNox ETHDILITHIUM implementation)
NIST KAT vectors (Dilithium)	PASS (standard Dilithium variant)
ETHDilithium gas-optimised variant	Functional (~6.6M gas, Keccak-based)
Falcon on-chain verification (FIPS 206)	Available (ZKNox ETHFALCON — not yet integrated into SuVa)
SPHINCS+ EVM implementation (FIPS 205)	Not yet started
Quantum-safe vault / wrapper contracts	Not yet started
Stablecoin integration middleware	Not yet started
Signing SDK / client tools	Not yet started
Formal security audit	Not yet performed

Item	Status
Production readiness	Experimental — DO NOT USE IN PRODUCTION
License	MIT (ZKNoxHQ)

10. Forks and Source Repositories

Repository	URL	Relationship
GiacomoPope/dilithium-py	https://github.com/GiacomoPope/dilithium-py/	Python reference is forked from here
ZKNoxHQ/NTT	https://github.com/ZKNoxHQ/NTT	NTT implementation source
ZKNoxHQ/ETHFALCON	https://github.com/ZKNoxHQ/ETHFALCON	Related post-quantum work (Falcon)
pq-crystals/dilithium	https://github.com/pq-crystals/dilithium/	Official NIST reference; KAT source
pq-crystals/security-estimates	https://github.com/pq-crystals/security-estimates	Lattice security estimates basis
zhenfeizhang/falcon-go	https://github.com/zhenfeizhang/falcon-go/blob/main/c/keccak_prng.c	Keccak PRNG basis
coruus/py-keccak	https://github.com/coruus/py-keccak/blob/master/fips202/keccak.py	SHAKE-256 Solidity translation source
usnistgov/ACVP-Server	https://github.com/usnistgov/ACVP-Server/releases/tag/v1.10.35	ML-DSA test vectors
foundry-rs/forge-std	https://github.com/foundry-rs/forge-std	Solidity testing framework

License: MIT. Copyright (c) 2025 ZKNoxHQ.

11. Technical Roadmap — What Needs To Be Built

Phase 1: Validate Existing Foundation (Current Stage)

Task	Description	Status
Academic review of Dilithium contracts	Verify that the Solidity implementation correctly follows FIPS 204	Not started
Security parameter validation	Confirm ETHDilithium (modified hash, precomputed NTT) maintains NIST Level 2 security	Not started
NTT assembly audit	Review Yul assembly for correctness and gas efficiency	Not started
Formal verification	Apply formal methods to prove NTT correctness	Not started

Phase 2: Integrate Falcon (Primary) and SPHINCS+ (Fallback)

Task	Description	Status
Falcon EVM integration	Integrate ZKNox's existing ETHFALCON implementation (~350K gas) as the primary on-chain verifier. Falcon (FN-DSA / FIPS 206) is lattice-based (NTRU) and ~20x cheaper than Dilithium on EVM.	Not started
Falcon academic review	Validate the ETHFALCON implementation against the FIPS 206 specification	Not started

Task	Description	Status
SPHINCS+ EVM implementation	Implement hash-based post-quantum signatures on-chain as the "nuclear bunker" fallback. SPHINCS+ (SLH-DSA / FIPS 205) relies only on hash functions — the most conservative security assumption.	Not started
SPHINCS+ gas optimisation	Optimise for EVM; explore Keccak-native variant and L2 deployment where gas costs are 10-100x lower	Not started
Dual-scheme architecture	Design the protocol so users can choose Falcon (cheap, daily use) or SPHINCS+ (expensive, maximum security) per transaction	Not started

Rationale for dual approach:

Scheme	Role	Security basis	Risk profile
Falcon	Daily transactions	Lattice (NTRU) — if lattice math breaks, Falcon breaks	Low risk, but lattice cryptography is ~20 years old
SPHINCS+	High-value / long-term protection	Hash functions (SHA/SHAKE) — only breakable if hash functions break	Lowest risk — hash functions studied for 40+ years, no serious breaks

If a breakthrough ever compromises lattice-based cryptography, all Falcon and Dilithium signatures would be at risk — but SPHINCS+ would remain secure. This defence-in-depth strategy is a key differentiator for SuVa.

Phase 3: Build the SuVa Protocol Layer

Task	Description	Status
Quantum-safe vault contracts	Smart contracts that hold stablecoins (USDT, USDC, etc.) and require Falcon (primary) or SPHINCS+ (fallback) signatures to authorise withdrawals and transfers	Not started
Wrapped token contracts (qUSDT, qUSDC)	ERC-20 tokens representing quantum-protected stablecoins, minted when users deposit into the vault	Not started
Signing tool / SDK	Client-side tool for users to generate Falcon/SPHINCS+ keypairs and sign transactions (browser extension, mobile app, or CLI)	Not started
Integration API for issuers	Interface allowing stablecoin issuers to integrate quantum-safe verification into their existing contracts	Not started
Documentation for integrators	Technical docs, API reference, integration guides	Not started
Deployment scripts for mainnet/L2	Production deployment tooling for Ethereum mainnet and L2 chains	Minimal

Phase 4: Audit, Certify, and Launch

Task	Description	Status
Professional security audit	Engage a recognised firm (Trail of Bits, NCC Group, or equivalent)	Not started

Task	Description	Status
Testnet deployment and public testing	Deploy on Ethereum testnets (Sepolia, Goerli) and L2 testnets	Not started
Mainnet deployment	Production launch	Not started

12. EU Regulatory Roadmap

For SuVa to operate as a stablecoin infrastructure service in Europe, the following regulatory steps are required:

12.1 MiCA Compliance (Markets in Crypto-Assets Regulation)

MiCA came into full effect in the EU in 2024. Key requirements for SuVa:

Requirement	What it means for SuVa
E-Money Token (EMT) licence	If SuVa issues wrapped stablecoins (qUSDT, qUSDC), it may need an EMT licence or must partner with a licensed issuer
Asset-Referenced Token rules	Wrapped tokens pegged to existing stablecoins must comply with reserve and transparency requirements
CASP licence	If SuVa provides custody services (holding stablecoins in vaults), it needs a Crypto-Asset Service Provider licence
Whitepaper requirements	A regulatory-compliant whitepaper must be filed with the relevant EU authority

12.2 Steps to EU Market

- Legal entity** — Establish a company in an EU jurisdiction (France, Ireland, or Germany recommended)
- Legal opinion** — Obtain legal counsel on whether qUSDT/qUSDC constitutes an EMT under MiCA
- Licence application** — Apply for CASP and/or EMT licence through the relevant National Competent Authority

4. **Reserve management** — If issuing wrapped tokens, establish compliant reserve management
5. **Ongoing compliance** — Regular audits, reporting, and capital requirements

12.3 Estimated Costs

Item	Estimated Cost
Legal counsel (MiCA compliance)	EUR 50,000 - 150,000
CASP licence application	EUR 20,000 - 50,000
EMT licence (if needed)	EUR 100,000 - 350,000
Professional security audit	EUR 150,000 - 350,000
Total estimated regulatory costs	EUR 300,000 - 900,000

13. Self-Funding via \$SVA Token

To fund development, audits, and regulatory compliance, SuVa plans to launch a governance and utility token (\$SVA). This is a well-established model in academic blockchain projects:

13.1 Academic Precedents

Project	Token	Academic Origin	What happened
Algorand	ALGO	MIT (Silvio Micali, Turing Award winner)	Raised \$60M via token sale; now a top-50 blockchain
Cardano	ADA	IOHK / University of Edinburgh	Peer-reviewed academic research; top-10 blockchain
Zcash	ZEC	Johns Hopkins / MIT	Academic cryptography (zero-knowledge proofs); launched via token

Project	Token	Academic Origin	What happened
Starknet	STRK	Technion (Eli Ben-Sasson)	Academic ZK-STARK research; \$8B valuation
Mina Protocol	MINA	UC Berkeley	Succinct blockchain research; token-funded

These projects demonstrate that rigorous academic research, combined with token-based funding, can produce globally significant blockchain infrastructure.

13.2 \$SVA Token Purpose

- **Governance** — Token holders participate in protocol decisions
 - **Gas fee subsidies** — \$SVA can be used to subsidise the higher gas costs of post-quantum verification
 - **Staking** — Validators and infrastructure operators stake \$SVA
 - **Self-funding** — Token sale proceeds fund audits, licences, and development
-

14. What's In It For Academic Collaborators

We are seeking professors and researchers who wish to participate in the development and validation of SuVa. Below are the advisory and collaboration roles available. We welcome suggestions for other arrangements.

14.1 Advisory Roles

Role	Description	Commitment
Cryptographic Advisor	Review and validate the post-quantum signature implementations (Dilithium, SPHINCS+). Confirm correctness against NIST specifications.	Low — review documentation and code, provide written feedback
Research Co-PI	Co-lead a grant application (e.g., Innovate UK, EPSRC, Horizon Europe) as a	Medium — co-write grant proposal, supervise research students

Role	Description	Commitment
	Co-Principal Investigator, with SuVa as the Industry Partner	
Formal Methods Advisor	Apply formal verification techniques to prove correctness of the assembly-level NTT and signature verification	Medium — research project, potential publication
Security Advisor	Assess the security assumptions, threat model, and parameter choices. Review ETHDilithium modifications.	Low-Medium — analysis and written assessment
Regulatory / Standards Advisor	Advise on alignment with EU MiCA regulations and NIST post-quantum standards	Low — periodic consultation

We are open to other advisory arrangements. If you see a different way your expertise could contribute, we would be delighted to discuss it.

14.2 Compensation and Incentives

Benefit	Details
Co-PI on grant applications	SuVa will serve as the Industry Partner on UKRI, EPSRC, or Horizon Europe grants. Academic partners can apply as PI or Co-PI.
Patent co-authorship	If novel techniques are developed (e.g., new gas optimisation methods, formal verification approaches), academic collaborators would be co-inventors on any patent filings.
Publication opportunities	Co-author academic papers on post-quantum EVM implementations, security analyses, or formal verification results.
Consulting fees	For substantive advisory work beyond informal guidance, consulting fees apply (to be agreed based on scope).

Benefit	Details
Token equity in \$SVA	Advisory board members receive token allocation in \$SVA (vesting terms to be discussed based on the advisory role and level of involvement).
Introductions and network	Anyone who can facilitate relevant introductions — to cryptographers, blockchain researchers, regulatory experts, or industry partners — is also highly valued.

14.3 What We Ask

1. **Review this document** — Does the technical description accurately represent the codebase? Are there obvious gaps or concerns?
2. **Review the code** — If interested, examine the Solidity and/or Python code and provide feedback on correctness and quality
3. **Guide the next steps** — Help us prioritise the roadmap (Phase 1-4) and direct our research team on what to build next
4. **Introductions** — Connect us with colleagues who may be interested in collaborating

Our research team will handle all development work. We are looking for academic guidance, validation, and direction.

15. References

- **FIPS 204** — Module-Lattice-Based Digital Signature Standard (ML-DSA / Dilithium). NIST, August 2024.
 - **FIPS 205** — Stateless Hash-Based Digital Signature Standard (SLH-DSA / SPHINCS+). NIST, August 2024.
 - **FIPS 206** — FFT (Fast Fourier Transform) over NTRU-Lattice-Based Digital Signature Standard (FN-DSA / Falcon). NIST, expected 2025.
 - **CNSA 2.0** — NSA Commercial National Security Algorithm Suite 2.0. Mandates post-quantum transition by 2030-2035.
 - **ERC-4337** — Account Abstraction Using Alt Mempool (Ethereum standard). Enables smart contract wallets with custom signature verification.
 - **MiCA** — Markets in Crypto-Assets Regulation (EU 2023/1114). Full enforcement from June 2024.
-

16. Contact

William Schulz UCL Alumni 2023 www.suva.tech @suvaproto

This document was prepared for academic review and collaboration purposes. The codebase described herein is experimental and has not been audited for production use. We welcome all feedback, corrections, and suggestions.